Lab Assignment 6

Topics:

1. Array of objects, Constructors in array of objects
2. Pointer to objects, this pointer
3. Passing and returning objects
4. Static and const member function
5. Inline function

1. Implement a class ***Library*** that contains **an array of *Book objects***. The ***Book*** class should have attributes like **title, author, and ISBN**. The ***Library*** class should provide following methods:
    a. **bool addNewBook(title,author,ISBN)** to add the new book in the book list. The return type should be Boolean to check whether the book is added or not. This function should take the argument using pass-by-reference method. The attribute of the class (title, author, and ISBN) and the function arguments of function **addNewBook** should be same. Use the **scope resolution operator** to access the class data members.
    b. **bool removeBooks(ISBN)** to remove the book using the input parameter ISBN. The return type should be Boolean to check whether the book is removed or not. This function also should take the argument using pass-by- reference method. Define function outside the class using scope resolution operator.
    c. **displayDetails()** to list the books.

Write a main function to demonstrate adding, removing, and displaying books from the library. Add at least 5 books to demonstrate the adding book, 1 book to remove from the list.

CODE:

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Book {
public:
    string title;
    string author;
    int ISBN;

    Book(string t=" ", string a=" ", int i=0) {
        title = t;
        author = a;
        ISBN = i;
    }
};

class Library {
private:
    Book books[100];
    int count;

public:
    Library() {
        count = 0;
    }

    bool addNewBook(string &title, string &author, int &ISBN) {
        if (count >= 100) {
            return false;
        }

        books[count].title = title;
        books[count].author = author;
        books[count].ISBN = ISBN;

        count++;
```

```
        return true;
    }

    bool removeBooks(int &ISBN);

    void displayDetails() {
        if (count == 0) {
            cout << "No books in library.\n";
            return;
        }

        cout << "Library Books:\n";
        for (int i = 0; i < count; i++) {
            cout << "Title: " << books[i].title << endl;
            cout << "Author: " << books[i].author << endl;
            cout << "ISBN: " << books[i].ISBN << endl;
        }
    }
};

bool Library::removeBooks(int &ISBN) {
    for (int i = 0; i < count; i++) {
        if (books[i].ISBN == ISBN) {

            for (int j = i; j < count - 1; j++) {
                books[j] = books[j + 1];
            }

            count--;
            return true;
        }
    }
    return false;
}
```

```
int main() {
    Library lib;

    string t1 = "C++", a1 = "Shagun"; int i1 = 101;
    string t2 = "DSA", a2 = "Mark Allen"; int i2 = 102;
    string t3 = "Operating System", a3 = "Harshit"; int i3 = 103;
    string t4 = "Computer Networks", a4 = "Kiran"; int i4 = 104;
    string t5 = "DBMS", a5 = "Vikas"; int i5 = 105;

    lib.addNewBook(t1, a1, i1);
    lib.addNewBook(t2, a2, i2);
    lib.addNewBook(t3, a3, i3);
    lib.addNewBook(t4, a4, i4);
    lib.addNewBook(t5, a5, i5);

    lib.displayDetails();
    int removeISBN = 103;

    if (lib.removeBooks(removeISBN)) {
        cout << "\nBook removed successfully.\n";
    } else {
        cout << "Book not found.\n";
    }

    lib.displayDetails();

    return 0;
}
```

```
PS C:\Users\Shagun Garg> cd "c:\Users\Shagun Garg\" ; if ($?) { g++ c
Library Books:
Title: C++
Author: Shagun
ISBN: 101
Title: DSA
Author: Mark Allen
ISBN: 102
Title: Operating System
Author: Harshit
ISBN: 103
Title: Computer Networks
Author: Kiran
ISBN: 104
Title: DBMS
Author: Vikas
ISBN: 105

Book removed successfully.
Library Books:
Title: C++
Author: Shagun
ISBN: 101
Title: DSA
Author: Mark Allen
ISBN: 102
Title: Computer Networks
Author: Kiran
ISBN: 104
Title: DBMS
Author: Vikas
ISBN: 105
PS C:\Users\Shagun Garg>
```

2. Implement the question 1 using constructor. Do the following modification:
    a. Add default constructor, parameterized constructor, and copy constructor to add the details of the book using the same parameters, i.e., **title, author, and ISBN.**
    b. Use the "this" pointer instead of scope resolution operator to access the data members of the class inside the constructor and function definition.
    c. Create function **bool removeBooks(ISBN)** same as the question 1**.**
    d. **displayDetails()** same as question 1.
    e. Create array of objects using 1. Initializer list, 2. Dynamic initialization
3. Create a class ***Account*** with attributes ***account number (of const long type), transaction ID (of long type), transaction type (credit/debit) (of string type), and balance (of double type).*** The Account class should provide following methods:
    a. **long depositAmount(to, from, amount)**. The "to" and "from" are the account number of the sender and receiver. The function should return the transaction id.
    b. **long creditAmount(to, from, amount)** similar to the depositAmount function. This function should also return the transaction id.
    c. **void displayDetails()** should display all the details such as account number, remaining balance, transaction history (credited/debited). Make this function as the const function.
    d. In all the functions, make all the arguments as the const arguments.

Write a main function to demonstrate all the function calls. Create at least 5 accounts objects (using array or without array) to demonstrate the operations of credit and debit.

CODE:

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Account {
private:
    const long accountNumber;
    long transactionID;
    string transactionType;
    double balance;

    static long nextTransactionID;

public:

    Account(long accNo, double bal) : accountNumber(accNo), balance(bal) {
        transactionID = 0;
        transactionType = "None";
    }

    long depositAmount(const long to, const long from, const double amount) {
        if (accountNumber == from) {
            if (balance >= amount) {
                balance -= amount;
                transactionID = ++nextTransactionID;
                transactionType = "Debit";
                cout << "Amount " << amount << " debited from Account " << from <<
endl;
                cout << "Transferred to Account " << to << endl;
                return transactionID;
            } else {
                cout << "Insufficient Balance in Account " << from << endl;
                return -1;
            }
        }
        return -1;
    }
```

```
    long creditAmount(const long to, const long from, const double amount) {
        if (accountNumber == to) {
            balance += amount;
            transactionID = ++nextTransactionID;
            transactionType = "Credit";
            cout << "Amount " << amount << " credited to Account " << to << endl;
            cout << "Received from Account " << from << endl;
            return transactionID;
        }
        return -1;
    }

    void displayDetails() const {
        cout << "\nAccount Number: " << accountNumber << endl;
        cout << "Balance: " << balance << endl;
        cout << "Last Transaction ID: " << transactionID << endl;
        cout << "Last Transaction Type: " << transactionType << endl;
    }

    long getAccountNumber() const {
        return accountNumber;
    }
};

long Account::nextTransactionID = 1000;

int main() {
    const int n = 5;
    long accNo;
    double bal;

    Account* accounts[n];
```

```
cout << "Enter details for 5 accounts:\n";
for (int i = 0; i < n; i++) {
    cout << "\nEnter Account Number: ";
    cin >> accNo;
    cout << "Enter Initial Balance: ";
    cin >> bal;

    accounts[i] = new Account(accNo, bal);
}

long from, to;
double amount;

cout << "\nEnter sender account number: ";
cin >> from;
cout << "Enter receiver account number: ";
cin >> to;
cout << "Enter amount: ";
cin >> amount;

long tid = -1;
for (int i = 0; i < n; i++) {
    tid = accounts[i]->depositAmount(to, from, amount);
    if (tid != -1)
        break;
}
if (tid != -1) {
    for (int i = 0; i < n; i++) {
        accounts[i]->creditAmount(to, from, amount);
    }
    cout << "\nTransaction Successful! Transaction ID: " << tid << endl;
}
```

```
    cout << "\n--- Account Details ---\n";
    for (int i = 0; i < n; i++) {
        accounts[i]->displayDetails();
    }
    for (int i = 0; i < n; i++) {
        delete accounts[i];
    }

    return 0;
}
```

INPUTS:

```
PS C:\Users\Shagun Garg> cd "c:\Users\Shagun Garg\" ;
Enter details for 5 accounts:

Enter Account Number: 12345
Enter Initial Balance: 12000

Enter Account Number: 23456
Enter Initial Balance: 78000

Enter Account Number: 34567
Enter Initial Balance: 16000

Enter Account Number: 56789
Enter Initial Balance: 34900

Enter Account Number: 45678
Enter Initial Balance: 23000

Enter sender account number: 12345
Enter receiver account number: 23456
Enter amount: 10000
Amount 10000 debited from Account 12345
Transferred to Account 23456
Amount 10000 credited to Account 23456
Received from Account 12345

Transaction Successful! Transaction ID: 1001
```

OUTPUTS:

```
--- Account Details ---

Account Number: 12345
Balance: 2000
Last Transaction ID: 1001
Last Transaction Type: Debit

Account Number: 23456
Balance: 88000
Last Transaction ID: 1002
Last Transaction Type: Credit

Account Number: 34567
Balance: 16000
Last Transaction ID: 0
Last Transaction Type: None

Account Number: 56789
Balance: 34900
Last Transaction ID: 0
Last Transaction Type: None

Account Number: 45678
Balance: 23000
Last Transaction ID: 0
Last Transaction Type: None
PS C:\Users\Shagun Garg>
```

4. Write a program to add data objects of two different classes using friend functions.

```
#include <iostream>
using namespace std;

class ClassB;
class ClassA {
private:
    int a;
public:
    ClassA(int x) {
        a = x;
    }
    friend int add(ClassA, ClassB);
};

```

```
class ClassB {
private:
    int b;
public:
    ClassB(int y) {
        b = y;
    }
    friend int add(ClassA, ClassB);
};
int add(ClassA obj1, ClassB obj2) {
    return obj1.a + obj2.b;
}

int main() {
    int x, y;
    cout << "Enter value for ClassA: ";
    cin >> x;
    cout << "Enter value for ClassB: ";
    cin >> y;
    ClassA obj1(x);
    ClassB obj2(y);
    int result = add(obj1, obj2);
    cout << "Sum of two objects = " << result << endl;
    return 0;
}
```

```
Enter value for ClassA: 45
Enter value for ClassB: 60
Sum of two objects = 105
PS C:\Users\Shagun Garg>
```

44. Define a class named ***Complex*** with properties (real and imaginary) and methods as per following details.

    a. ***Parameterized constructor*** and ***copy constructor*** to initialize object values.

    b. ***void display ()*** to display complex number.

    c. ***void sum (Complex, Complex)*** to add two complex numbers (objects of Complex class). Implement this function as the friend function.

```
#include <iostream>
using namespace std;

class Complex {
private:
   double real;
   double imag;

public:
   // Parameterized Constructor
   Complex(double r, double i) {
      real = r;
      imag = i;
   }

   // Copy Constructor
   Complex(const Complex &c) {
      real = c.real;
      imag = c.imag;
   }
   void display() const {
      cout << real;
      if (imag >= 0)
         cout << " + " << imag << "i";
      else
         cout << " - " << -imag << "i";
      cout << endl;
   }
 friend void sum(Complex, Complex);
};

// Friend Function Definition
```

```
void sum(Complex c1, Complex c2) {
   Complex result(0, 0);

   result.real = c1.real + c2.real;
   result.imag = c1.imag + c2.imag;

   cout << "Sum of Complex Numbers = ";
   result.display();
}
int main() {
   double r1, i1, r2, i2;

   cout << "Enter real and imaginary part of first complex
number: ";
   cin >> r1 >> i1;

   cout << "Enter real and imaginary part of second complex
number: ";
   cin >> r2 >> i2;

   // Parameterized constructor
   Complex c1(r1, i1);
   Complex c2(r2, i2);

   // Copy constructor
   Complex c3 = c1;

   cout << "\nFirst Complex Number: ";
   c1.display();

   cout << "Second Complex Number: ";
   c2.display();
```

```
    cout << "Copied Complex Number (from first): ";
    c3.display();

    // Friend function call
    sum(c1, c2);

    return 0;
}
```

```
PS C:\Users\Shagun Garg> cd "c:\Users\Shagun Garg\" ; if ($?) {
Enter real and imaginary part of first complex number: 23
45
Enter real and imaginary part of second complex number: 67
11

First Complex Number: 23 + 45i
Second Complex Number: 67 + 11i
Copied Complex Number (from first): 23 + 45i
Sum of Complex Numbers = 90 + 56i
PS C:\Users\Shagun Garg>
```

==========********==========